

Lecture Summary: Python OOP – Inheritance and Polymorphism

Introduction This lecture focuses on advanced Object-Oriented Programming (OOP) concepts in Python, specifically inheritance and polymorphism. These mechanisms are essential for building code that is reusable, scalable, and easy to maintain.

1. Inheritance Inheritance allows a "Child Class" to derive properties and behaviors from a "Parent Class." This promotes code reuse and hierarchical organization.

- **Single Inheritance:** A child class inherits from a single parent class.
- **Multiple Inheritance:** A child class inherits features from more than one parent class.
- **Multi-level Inheritance:** A derived class inherits from another derived class (creating a chain like Grandparent -> Parent -> Child).
- **The `super()` Function:** This is used to call methods from the parent class inside the child class, ensuring that the parent's logic is executed alongside the child's extensions.

2. Polymorphism Polymorphism refers to the ability of different objects to respond to the same function call in their own specific ways.

- **Method Overriding:** A child class provides its own specific implementation of a method that is already defined in the parent class.
- **Built-in Polymorphism:** Examples include the `len()` function, which can calculate the length of various data types (strings, lists, dictionaries) regardless of their internal structure.

3. Special (Dunder) Methods Dunder methods (Double Underscore methods) allow custom objects to interact with Python's built-in operations.

- **`__init__`:** Used for object initialization when a new instance is created.
- **`__str__`:** Defines the string representation of an object (what shows up when you use `print()`).
- **`__add__`:** Allows the use of the `+` operator to define how two objects should be added together.
- **`__len__`:** Customizes the behavior of the `len()` function for your objects.
- **`__eq__`:** Defines the logic for comparing two objects using the equality operator (`==`).